Incident Recovery Report: baseline-rules

Agent: **rules** · 22 incidents · generated 2026-10-30 03:16 UTC

How to read this report

Each incident was injected into a live five-service system. The agent investigated and acted only through a guarded toolbox, then filed a root-cause report. An independent verifier sent real user traffic over a soak window and checked that each fault's root cause was actually gone. Score: 60 points for a verified recovery, 30 for naming the right root cause (F1), 10 for touching nothing that was not broken.

Summary

SPLIT	INCIDENTS	RECOVERED	DIAGNOSIS F1	MEAN SCORE	COLLATERAL
dev	10	100%	0.97	99.0	0
held-out	12	25%	0.35	35.4	0

Incidents

INCIDENT	SPLIT	RESULT	SCORE	REPORTED ROOT CAUSE
dev-01-api-down	dev	recovered	100	api/service_down
dev-02-postgres-down	dev	recovered	100	postgres/service_down
dev-03-redis-down	dev	recovered	100	redis/service_down
dev-04-db-password	dev	recovered	100	api/bad_config_deploy
dev-05-gateway-upstream	dev	recovered	100	gateway/bad_config_deploy
dev-06-disk-full	dev	recovered	100	api/disk_full
dev-07-cpu-hog	dev	recovered	100	api/cpu_exhaustion
dev-08-postgres-partition	dev	recovered	100	postgres/network_partition
dev-09-multi-disk-password	dev	recovered	90	api/bad_config_deploy
dev-10-multi-redis-gateway	dev	recovered	100	redis/service_down; gateway/bad_config_deploy
ho-01-db-host-dns	held-out	not recovered	10	none
ho-02-crashloop	held-out	not recovered	10	none
ho-03-partial-tax-rate	held-out	not recovered	10	none
ho-04-memory-leak	held-out	not recovered	10	none
ho-05-db-lock	held-out	not recovered	10	none
ho-06-conn-exhaustion	held-out	not recovered	10	none
ho-07-redis-partition	held-out	recovered	100	redis/network_partition
ho-08-rollback-trap	held-out	recovered	100	api/bad_config_deploy
ho-09-lock-behind-gateway	held-out	not recovered	30	gateway/bad_config_deploy
ho-10-cpu-and-redis-partition	held-out	recovered	100	redis/network_partition; api/cpu_exhaustion
ho-11-conn-leak-and-dns	held-out	not recovered	10	none
ho-12-triple	held-out	not recovered	25	redis/service_down

dev-01-api-down DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2003ms over the last few seconds.
  - GET /api/health: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/products: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/orders/recent: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - POST /api/orders: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **service_stopped** on api: A core service process is stopped.

Root causes to name: **api/service_down**

Agent's incident report

• **api/service_down**: api was exited

Summary: runbook applied 1 fix(es)

Investigation and actions (12 tool calls, 1 state changes, 12.8 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	act	**start_service**(service=api)
3	1.1s	observe	**check_endpoints**()
4	1.1s	observe	**get_logs**(service=gateway, since_seconds=120, grep=upstream)
5	1.1s	observe	**get_recent_changes**()
6	1.1s	observe	**get_logs**(service=api, since_seconds=120)
7	1.1s	observe	**run_diagnostic**(service=api, check=disk)
8	1.3s	observe	**get_metrics**(service=api)
9	3.5s	observe	**list_services**()
10	3.6s	observe	**check_endpoints**()
11	3.7s	observe	**verify_recovery**()
12	12.8s	final	**submit_report**(root_causes=[{'component': 'api', 'category': 'service_down', 'evidence': 'api was exited'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	46 ms	0	1.00

dev-02-postgres-down DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 8ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **service_stopped** on postgres: A core service process is stopped.
Root causes to name: **postgres/service_down**

Agent's incident report

• **postgres/service_down**: postgres was exited
Summary: runbook applied 1 fix(es)

Investigation and actions (10 tool calls, 1 state changes, 12.4 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	act	**start_service**(service=postgres)
3	0.5s	observe	**check_endpoints**()
4	0.5s	observe	**get_logs**(service=api, since_seconds=120)
5	0.6s	observe	**run_diagnostic**(service=api, check=disk)
6	0.7s	observe	**get_metrics**(service=api)
7	2.9s	observe	**list_services**()
8	3.1s	observe	**check_endpoints**()
9	3.2s	observe	**verify_recovery**()
10	12.4s	final	**submit_report**(root_causes=[{'component': 'postgres', 'category': 'service_down', 'evidence': 'postgres was exited'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	51 ms	0	1.00

dev-03-redis-down DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 29ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **service_stopped** on redis: A core service process is stopped.

Root causes to name: **redis/service_down**

Agent's incident report

• **redis/service_down**: redis was exited

Summary: runbook applied 1 fix(es)

Investigation and actions (5 tool calls, 1 state changes, 9.7 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	act	**start_service**(service=redis)
3	0.6s	observe	**check_endpoints**()
4	0.7s	observe	**verify_recovery**()
5	9.8s	final	**submit_report**(root_causes=[{'component': 'redis', 'category': 'service_down', 'evidence': 'redis was exited'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	48 ms	0	1.00

dev-04-db-password DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 29ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **bad_db_password** on api: Config deploy with wrong DB password.
Root causes to name: **api/bad_config_deploy**

Agent's incident report

• **api/bad_config_deploy**: db password authentication failed
Summary: runbook applied 1 fix(es)

Investigation and actions (10 tool calls, 1 state changes, 23.1 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.3s	observe	**get_logs**(service=api, since_seconds=120)
4	0.4s	act	**rollback_config**(service=api)
5	11.1s	observe	**run_diagnostic**(service=api, check=disk)
6	11.3s	observe	**get_metrics**(service=api)
7	13.5s	observe	**list_services**()
8	13.7s	observe	**check_endpoints**()
9	13.8s	observe	**verify_recovery**()
10	23.1s	final	**submit_report**(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': 'db password authentication failed'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	47 ms	0	1.00

dev-05-gateway-upstream DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
• Red herrings planted: benign_api_deploy
Root causes to name: **gateway/bad_config_deploy**

Agent's incident report

• **gateway/bad_config_deploy**: upstream connection refused after gateway deploy
Summary: runbook applied 1 fix(es)

Investigation and actions (12 tool calls, 1 state changes, 12.3 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.2s	observe	**get_logs**(service=gateway, since_seconds=120, grep=upstream)
4	0.2s	observe	**get_recent_changes**()
5	0.2s	act	**rollback_config**(service=gateway)
6	0.4s	observe	**get_logs**(service=api, since_seconds=120)
7	0.4s	observe	**run_diagnostic**(service=api, check=disk)
8	0.6s	observe	**get_metrics**(service=api)
9	2.9s	observe	**list_services**()
10	3.1s	observe	**check_endpoints**()
11	3.1s	observe	**verify_recovery**()
12	12.3s	final	**submit_report**(root_causes=[{'component': 'gateway', 'category': 'bad_config_deploy', 'evidence': 'upstream connection refused after gateway deploy'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	69 ms	0	1.00

dev-06-disk-full DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 31ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"ok","database":"ok","disk":"error: [Errno 28] No space left )
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **disk_full** on api: A runaway export fills the API data volume.
Root causes to name: **api/disk_full**

Agent's incident report

• **api/disk_full**: /data at 100%
Summary: runbook applied 1 fix(es)

Investigation and actions (10 tool calls, 1 state changes, 12.4 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.3s	observe	**get_logs**(service=api, since_seconds=120)
4	0.3s	observe	**run_diagnostic**(service=api, check=disk)
5	0.5s	act	**delete_file**(service=api, path=/data/tmp/export-orders-20261029.csv.tmp)
6	0.6s	observe	**get_metrics**(service=api)
7	2.8s	observe	**list_services**()
8	3.0s	observe	**check_endpoints**()
9	3.1s	observe	**verify_recovery**()
10	12.4s	final	**submit_report**(root_causes=[{'component': 'api', 'category': 'disk_full', 'evidence': '/data at 100%'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	41 ms	0	1.00

dev-07-cpu-hog DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 1399ms over the last few seconds.
  - all probes passing but p95 latency is 1399ms
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **cpu_hog** on api: A runaway search reindex job saturates the API's CPU.
Root causes to name: **api/cpu_exhaustion**

Agent's incident report

• **api/cpu_exhaustion**: python /app/tools/reindex_search.py hogging CPU
Summary: runbook applied 1 fix(es)

Investigation and actions (11 tool calls, 1 state changes, 19.0 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	2.1s	observe	**get_logs**(service=api, since_seconds=120)
4	2.1s	observe	**run_diagnostic**(service=api, check=disk)
5	3.7s	observe	**get_metrics**(service=api)
6	7.2s	observe	**run_diagnostic**(service=api, check=processes)
7	8.9s	act	**kill_process**(service=api, pid=23)
8	9.6s	observe	**list_services**()
9	9.8s	observe	**check_endpoints**()
10	9.9s	observe	**verify_recovery**()
11	19.0s	final	**submit_report**(root_causes=[{'component': 'api', 'category': 'cpu_exhaustion', 'evidence': 'python /app/tools/reindex_search.py hogging CPU'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	55 ms	0	1.00

dev-08-postgres-partition DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 8ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **network_partition** on postgres: A backend service is cut off from the backend network.
Root causes to name: **postgres/network_partition**

Agent's incident report

• **postgres/network_partition**: postgres missing from backend
Summary: runbook applied 1 fix(es)

Investigation and actions (5 tool calls, 1 state changes, 9.5 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.1s	act	**reconnect_network**(service=postgres, network=backend)
3	0.2s	observe	**check_endpoints**()
4	0.3s	observe	**verify_recovery**()
5	9.5s	final	**submit_report**(root_causes=[{'component': 'postgres', 'category': 'network_partition', 'evidence': 'postgres missing from backend'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	42 ms	0	1.00

dev-09-multi-disk-password DEV

RECOVERED score **90/100** = recovery 60 + diagnosis 20 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 36%, p95 30ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/products: 1/5 failed, statuses [200, 503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **disk_full** on api: A runaway export fills the API data volume.
• **bad_db_password** on api: Config deploy with wrong DB password.
Root causes to name: **api/disk_full; api/bad_config_deploy**

Agent's incident report

• **api/bad_config_deploy**: db password authentication failed
Summary: runbook applied 1 fix(es)

Investigation and actions (10 tool calls, 1 state changes, 22.9 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.3s	observe	**get_logs**(service=api, since_seconds=120)
4	0.3s	act	**rollback_config**(service=api)
5	11.1s	observe	**run_diagnostic**(service=api, check=disk)
6	11.4s	observe	**get_metrics**(service=api)
7	13.6s	observe	**list_services**()
8	13.8s	observe	**check_endpoints**()
9	13.8s	observe	**verify_recovery**()
10	22.9s	final	**submit_report**(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': 'db password authentication failed'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	46 ms	0	0.67

dev-10-multi-redis-gateway DEV

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **service_stopped** on redis: A core service process is stopped.
• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
• Red herrings planted: worker_warnings
Root causes to name: **redis/service_down; gateway/bad_config_deploy**

Agent's incident report

• **redis/service_down**: redis was exited
• **gateway/bad_config_deploy**: upstream connection refused after gateway deploy
Summary: runbook applied 2 fix(es)

Investigation and actions (13 tool calls, 2 state changes, 12.5 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	act	**start_service**(service=redis)
3	0.6s	observe	**check_endpoints**()
4	0.6s	observe	**get_logs**(service=gateway, since_seconds=120, grep=upstream)
5	0.6s	observe	**get_recent_changes**()
6	0.6s	act	**rollback_config**(service=gateway)
7	0.8s	observe	**get_logs**(service=api, since_seconds=120)
8	0.8s	observe	**run_diagnostic**(service=api, check=disk)
9	0.9s	observe	**get_metrics**(service=api)
10	3.2s	observe	**list_services**()
11	3.4s	observe	**check_endpoints**()
12	3.5s	observe	**verify_recovery**()
13	12.5s	final	**submit_report**(root_causes=[{'component': 'redis', 'category': 'service_down', 'evidence': 'redis was exited'}, {'component': 'gateway', 'category': 'bad_config_deploy', 'evidence': 'upstream connection refused after gateway deploy'}], summary=runbook applied 2 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	54 ms	0	1.00

ho-01-db-host-dns HELD-OUT

NOT RECOVERED score **10/100** = recovery 0 + diagnosis 0 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 9ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **bad_db_host** on api: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: **api/bad_config_deploy**

Agent's incident report

No root cause reported.
Summary: runbook applied 0 fix(es)

Investigation and actions (17 tool calls, 0 state changes, 15.3 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.2s	observe	**get_logs**(service=api, since_seconds=120)
4	0.3s	observe	**run_diagnostic**(service=api, check=disk)
5	0.4s	observe	**get_metrics**(service=api)
6	2.6s	observe	**list_services**()
7	2.8s	observe	**check_endpoints**()
8	2.8s	observe	**get_logs**(service=api, since_seconds=120)
9	2.8s	observe	**run_diagnostic**(service=api, check=disk)
10	3.0s	observe	**get_metrics**(service=api)
11	4.7s	observe	**list_services**()
12	4.8s	observe	**check_endpoints**()
13	4.9s	observe	**get_logs**(service=api, since_seconds=120)
14	4.9s	observe	**run_diagnostic**(service=api, check=disk)
15	5.0s	observe	**get_metrics**(service=api)
16	6.7s	observe	**verify_recovery**()
17	15.3s	final	**submit_report**(root_causes=[], summary=runbook applied 0 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
20%	7 ms	0	0.00

Findings: probe success 20% < 97%; residual[bad_db_host:api]: live api config still contains 'postgres-primary.internal'

ho-02-crashloop HELD-OUT

NOT RECOVERED score **10/100** = recovery 0 + diagnosis 0 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2002ms over the last few seconds.
  - GET /api/health: 3/3 failed, statuses [502, 504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/products: 3/3 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 3/3 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 3/3 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **crashloop_config** on api: Config deploy that makes the API crash on boot.

Root causes to name: **api/bad_config_deploy or api/service_down**

Agent's incident report

No root cause reported.

Summary: runbook applied 0 fix(es)

Investigation and actions (19 tool calls, 0 state changes, 32.5 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.2s	observe	**get_logs**(service=gateway, since_seconds=120, grep=upstream)
4	0.2s	observe	**get_recent_changes**()
5	0.2s	observe	**get_logs**(service=api, since_seconds=120)
6	0.5s	observe	**run_diagnostic**(service=api, check=disk)
7	0.5s	observe	**get_metrics**(service=api)
8	2.7s	observe	**list_services**()
9	2.9s	observe	**check_endpoints**()
10	6.9s	observe	**get_logs**(service=api, since_seconds=120)
11	6.9s	observe	**run_diagnostic**(service=api, check=disk)
12	7.1s	observe	**get_metrics**(service=api)
13	9.3s	observe	**list_services**()
14	9.5s	observe	**check_endpoints**()
15	17.5s	observe	**get_logs**(service=api, since_seconds=120)
16	17.6s	observe	**run_diagnostic**(service=api, check=disk)
17	17.6s	observe	**get_metrics**(service=api)
18	19.8s	observe	**verify_recovery**()
19	32.5s	final	**submit_report**(root_causes=[], summary=runbook applied 0 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
20%	2004 ms	0	0.00

Findings: probe success 20% < 97%; probe p95 2004ms > 400ms; api is restarting; residual[crashloop_config:api]: live api config still contains 'auditing:'

ho-03-partial-tax-rate HELD-OUT

NOT RECOVERED score **10/100** = recovery 0 + diagnosis 0 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 31ms over the last few seconds.
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **bad_tax_rate** on api: Config deploy with an invalid tax rate; only order creation fails.
Root causes to name: **api/bad_config_deploy**

Agent's incident report

No root cause reported.
Summary: runbook applied 0 fix(es)

Investigation and actions (17 tool calls, 0 state changes, 15.6 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.2s	observe	**get_logs**(service=api, since_seconds=120)
4	0.3s	observe	**run_diagnostic**(service=api, check=disk)
5	0.4s	observe	**get_metrics**(service=api)
6	2.6s	observe	**list_services**()
7	2.8s	observe	**check_endpoints**()
8	2.9s	observe	**get_logs**(service=api, since_seconds=120)
9	2.9s	observe	**run_diagnostic**(service=api, check=disk)
10	3.0s	observe	**get_metrics**(service=api)
11	4.7s	observe	**list_services**()
12	4.8s	observe	**check_endpoints**()
13	4.9s	observe	**get_logs**(service=api, since_seconds=120)
14	4.9s	observe	**run_diagnostic**(service=api, check=disk)
15	5.0s	observe	**get_metrics**(service=api)
16	6.7s	observe	**verify_recovery**()
17	15.6s	final	**submit_report**(root_causes=[], summary=runbook applied 0 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
80%	30 ms	0	0.00

Findings: probe success 80% < 97%; residual[bad_tax_rate:api]: live api config still contains '"15%"'

ho-04-memory-leak HELD-OUT

NOT RECOVERED score **10/100** = recovery 0 + diagnosis 0 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 90ms over the last few seconds.
  - all probes passing but p95 latency is 90ms
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **memory_leak** on api: Feature flag enabling an unbounded in-process cache (OOM loop).
Root causes to name: **api/bad_config_deploy or api/memory_exhaustion**

Agent's incident report

No root cause reported.
Summary: runbook applied 0 fix(es)

Investigation and actions (9 tool calls, 0 state changes, 13.4 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.3s	observe	**check_endpoints**()
3	0.8s	observe	**get_logs**(service=api, since_seconds=120)
4	1.3s	observe	**run_diagnostic**(service=api, check=disk)
5	1.4s	observe	**get_metrics**(service=api)
6	3.7s	observe	**list_services**()
7	3.8s	observe	**check_endpoints**()
8	4.0s	observe	**verify_recovery**()
9	13.3s	final	**submit_report**(root_causes=[], summary=runbook applied 0 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
91%	44 ms	0	0.00

Findings: probe success 91% < 97%; api restarted during soak (crash loop?); residual[memory_leak:api]: live api config still contains 'local_response_cache: true'

ho-05-db-lock HELD-OUT

NOT RECOVERED score **10/100** = recovery 0 + diagnosis 0 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 2026ms over the last few seconds.
  - GET /api/orders/recent: 1/1 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 1/1 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **db_lock** on postgres: A stuck migration holds an exclusive lock on the orders table.
Root causes to name: **postgres/db_lock_contention**

Agent's incident report

No root cause reported.
Summary: runbook applied 0 fix(es)

Investigation and actions (17 tool calls, 0 state changes, 30.5 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	4.2s	observe	**get_logs**(service=api, since_seconds=120)
4	4.3s	observe	**run_diagnostic**(service=api, check=disk)
5	4.4s	observe	**get_metrics**(service=api)
6	6.7s	observe	**list_services**()
7	6.9s	observe	**check_endpoints**()
8	10.9s	observe	**get_logs**(service=api, since_seconds=120)
9	11.0s	observe	**run_diagnostic**(service=api, check=disk)
10	11.1s	observe	**get_metrics**(service=api)
11	13.3s	observe	**list_services**()
12	13.5s	observe	**check_endpoints**()
13	17.6s	observe	**get_logs**(service=api, since_seconds=120)
14	17.6s	observe	**run_diagnostic**(service=api, check=disk)
15	17.8s	observe	**get_metrics**(service=api)
16	20.0s	observe	**verify_recovery**()
17	30.5s	final	**submit_report**(root_causes=[], summary=runbook applied 0 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
60%	2027 ms	0	0.00

Findings: probe success 60% < 97%; probe p95 2027ms > 400ms; residual[db_lock:postgres]: migration-runner sessions: 1

ho-06-conn-exhaustion HELD-OUT

NOT RECOVERED score **10/100** = recovery 0 + diagnosis 0 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 32ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **conn_exhaustion** on worker: A backfill job in the worker leaks DB connections (and reconnects).
Root causes to name: **worker/db_connection_exhaustion or postgres/db_connection_exhaustion**

Agent's incident report

No root cause reported.
Summary: runbook applied 0 fix(es)

Investigation and actions (17 tool calls, 0 state changes, 16.1 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.3s	observe	**get_logs**(service=api, since_seconds=120)
4	0.3s	observe	**run_diagnostic**(service=api, check=disk)
5	0.4s	observe	**get_metrics**(service=api)
6	2.7s	observe	**list_services**()
7	2.9s	observe	**check_endpoints**()
8	2.9s	observe	**get_logs**(service=api, since_seconds=120)
9	3.0s	observe	**run_diagnostic**(service=api, check=disk)
10	3.1s	observe	**get_metrics**(service=api)
11	4.8s	observe	**list_services**()
12	5.0s	observe	**check_endpoints**()
13	5.1s	observe	**get_logs**(service=api, since_seconds=120)
14	5.2s	observe	**run_diagnostic**(service=api, check=disk)
15	5.3s	observe	**get_metrics**(service=api)
16	6.8s	observe	**verify_recovery**()
17	16.1s	final	**submit_report**(root_causes=[], summary=runbook applied 0 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
20%	60 ms	0	0.00

Findings: probe success 20% < 97%; residual[conn_exhaustion:worker]: 'report_backfill' still running in worker; backfill connections=37

ho-07-redis-partition HELD-OUT

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 37ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **network_partition** on redis: A backend service is cut off from the backend network.
Root causes to name: **redis/network_partition**

Agent's incident report

• **redis/network_partition**: redis missing from backend
Summary: runbook applied 1 fix(es)

Investigation and actions (5 tool calls, 1 state changes, 9.5 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	act	**reconnect_network**(service=redis, network=backend)
3	0.3s	observe	**check_endpoints**()
4	0.4s	observe	**verify_recovery**()
5	9.5s	final	**submit_report**(root_causes=[{'component': 'redis', 'category': 'network_partition', 'evidence': 'redis missing from backend'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	50 ms	0	1.00

ho-08-rollback-trap HELD-OUT

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 48ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/products: 5/5 failed, statuses [503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **bad_db_password** on api: Config deploy with wrong DB password.
• Red herrings planted: benign_api_deploy
Root causes to name: **api/bad_config_deploy**

Agent's incident report

• **api/bad_config_deploy**: db password authentication failed
Summary: runbook applied 1 fix(es)

Investigation and actions (16 tool calls, 2 state changes, 36.4 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.3s	observe	**get_logs**(service=api, since_seconds=120)
4	0.3s	act	**rollback_config**(service=api)
5	11.1s	observe	**run_diagnostic**(service=api, check=disk)
6	11.3s	observe	**get_metrics**(service=api)
7	13.6s	observe	**list_services**()
8	13.7s	observe	**check_endpoints**()
9	13.8s	observe	**get_logs**(service=api, since_seconds=120)
10	13.8s	act	**rollback_config**(service=api)
11	24.5s	observe	**run_diagnostic**(service=api, check=disk)
12	24.8s	observe	**get_metrics**(service=api)
13	27.1s	observe	**list_services**()
14	27.2s	observe	**check_endpoints**()
15	27.3s	observe	**verify_recovery**()
16	36.4s	final	**submit_report**(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': 'db password authentication failed'}], summary=runbook applied 1 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	57 ms	0	1.00

ho-09-lock-behind-gateway HELD-OUT

NOT RECOVERED score **30/100** = recovery 0 + diagnosis 20 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 3ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **db_lock** on postgres: A stuck migration holds an exclusive lock on the orders table.
• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
Root causes to name: **postgres/db_lock_contention; gateway/bad_config_deploy**

Agent's incident report

• **gateway/bad_config_deploy**: upstream connection refused after gateway deploy
Summary: runbook applied 1 fix(es)

Investigation and actions (20 tool calls, 1 state changes, 26.6 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.2s	observe	**get_logs**(service=gateway, since_seconds=120, grep=upstream)
4	0.3s	observe	**get_recent_changes**()
5	0.3s	act	**rollback_config**(service=gateway)
6	0.4s	observe	**get_logs**(service=api, since_seconds=120)
7	0.4s	observe	**run_diagnostic**(service=api, check=disk)
8	0.6s	observe	**get_metrics**(service=api)
9	2.8s	observe	**list_services**()
10	3.0s	observe	**check_endpoints**()
11	7.0s	observe	**get_logs**(service=api, since_seconds=120)
12	7.1s	observe	**run_diagnostic**(service=api, check=disk)
13	7.2s	observe	**get_metrics**(service=api)
14	9.4s	observe	**list_services**()
15	9.5s	observe	**check_endpoints**()
16	13.6s	observe	**get_logs**(service=api, since_seconds=120)
17	13.7s	observe	**run_diagnostic**(service=api, check=disk)
18	13.8s	observe	**get_metrics**(service=api)
19	16.0s	observe	**verify_recovery**()

#	TIME	TYPE	TOOL CALL
20	26.6s	final	**submit_report**(root_causes=[{'component': 'gateway', 'category': 'bad_config_deploy', 'evidence': 'upstream connection refused after gateway deploy'}], summary=runbook applied 1 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
60%	2059 ms	0	0.67

Findings: probe success 60% < 97%; probe p95 2059ms > 400ms; residual[db_lock:postgres]: migration-runner sessions: 1

ho-10-cpu-and-redis-partition HELD-OUT

RECOVERED score **100/100** = recovery 60 + diagnosis 30 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 1087ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **cpu_hog** on api: A runaway search reindex job saturates the API's CPU.
• **network_partition** on redis: A backend service is cut off from the backend network.
• Red herrings planted: benign_gateway_deploy
Root causes to name: **api/cpu_exhaustion; redis/network_partition**

Agent's incident report

• **redis/network_partition**: redis missing from backend
• **api/cpu_exhaustion**: python /app/tools/reindex_search.py hogging CPU
Summary: runbook applied 2 fix(es)

Investigation and actions (12 tool calls, 2 state changes, 19.7 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	act	**reconnect_network**(service=redis, network=backend)
3	0.3s	observe	**check_endpoints**()
4	2.6s	observe	**get_logs**(service=api, since_seconds=120)
5	2.6s	observe	**run_diagnostic**(service=api, check=disk)
6	4.1s	observe	**get_metrics**(service=api)
7	7.7s	observe	**run_diagnostic**(service=api, check=processes)
8	9.1s	act	**kill_process**(service=api, pid=23)
9	10.4s	observe	**list_services**()
10	10.5s	observe	**check_endpoints**()
11	10.6s	observe	**verify_recovery**()
12	19.7s	final	**submit_report**(root_causes=[{'component': 'redis', 'category': 'network_partition', 'evidence': 'redis missing from backend'}, {'component': 'api', 'category': 'cpu_exhaustion', 'evidence': 'python /app/tools/reindex_search.py hogging CPU'}], summary=runbook applied 2 fix(es), recovered=True)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
100%	46 ms	0	1.00

ho-11-conn-leak-and-dns HELD-OUT

NOT RECOVERED score **10/100** = recovery 0 + diagnosis 0 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 36%, p95 10ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/products: 1/5 failed, statuses [200, 503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

• **conn_exhaustion** on worker: A backfill job in the worker leaks DB connections (and reconnects).
• **bad_db_host** on api: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: **worker/db_connection_exhaustion or postgres/db_connection_exhaustion; api/bad_config_deploy**

Agent's incident report

No root cause reported.
Summary: runbook applied 0 fix(es)

Investigation and actions (17 tool calls, 0 state changes, 15.4 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	observe	**check_endpoints**()
3	0.2s	observe	**get_logs**(service=api, since_seconds=120)
4	0.3s	observe	**run_diagnostic**(service=api, check=disk)
5	0.4s	observe	**get_metrics**(service=api)
6	2.6s	observe	**list_services**()
7	2.8s	observe	**check_endpoints**()
8	2.8s	observe	**get_logs**(service=api, since_seconds=120)
9	2.9s	observe	**run_diagnostic**(service=api, check=disk)
10	3.0s	observe	**get_metrics**(service=api)
11	4.7s	observe	**list_services**()
12	4.8s	observe	**check_endpoints**()
13	4.8s	observe	**get_logs**(service=api, since_seconds=120)
14	4.9s	observe	**run_diagnostic**(service=api, check=disk)
15	5.0s	observe	**get_metrics**(service=api)
16	6.7s	observe	**verify_recovery**()
17	15.4s	final	**submit_report**(root_causes=[], summary=runbook applied 0 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
20%	7 ms	0	0.00

Findings: probe success 20% < 97%; residual[conn_exhaustion:worker]: 'report_backfill' still running in worker; backfill connections=37; residual[bad_db_host:api]: live api config still contains 'postgres-primary.internal'

ho-12-triple HELD-OUT

NOT RECOVERED score **25/100** = recovery 0 + diagnosis 15 + safety 10

Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 32ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

What was actually broken

- **service_stopped** on redis: A core service process is stopped.
- **disk_full** on api: A runaway export fills the API data volume.
- **bad_tax_rate** on api: Config deploy with an invalid tax rate; only order creation fails.

Root causes to name: **redis/service_down; api/disk_full; api/bad_config_deploy**

Agent's incident report

- **redis/service_down**: redis was exited

Summary: runbook applied 1 fix(es)

Investigation and actions (18 tool calls, 1 state changes, 16.0 s)

#	TIME	TYPE	TOOL CALL
1	0.0s	observe	**list_services**()
2	0.2s	act	**start_service**(service=redis)
3	0.6s	observe	**check_endpoints**()
4	0.7s	observe	**get_logs**(service=api, since_seconds=120)
5	0.7s	observe	**run_diagnostic**(service=api, check=disk)
6	0.8s	observe	**get_metrics**(service=api)
7	3.1s	observe	**list_services**()
8	3.3s	observe	**check_endpoints**()
9	3.3s	observe	**get_logs**(service=api, since_seconds=120)
10	3.4s	observe	**run_diagnostic**(service=api, check=disk)
11	3.5s	observe	**get_metrics**(service=api)
12	5.1s	observe	**list_services**()
13	5.3s	observe	**check_endpoints**()
14	5.4s	observe	**get_logs**(service=api, since_seconds=120)
15	5.4s	observe	**run_diagnostic**(service=api, check=disk)
16	5.6s	observe	**get_metrics**(service=api)
17	7.1s	observe	**verify_recovery**()
18	16.0s	final	**submit_report**(root_causes=[{'component': 'redis', 'category': 'service_down', 'evidence': 'redis was exited'}], summary=runbook applied 1 fix(es), recovered=False)

Independent verification

USER TRAFFIC SUCCESS	P95 LATENCY	COLLATERAL ACTIONS	DIAGNOSIS F1
80%	31 ms	0	0.50

Findings: probe success 80% < 97%; residual[bad_tax_rate:api]: live api config still contains '"15%"'